

Assignment 04

Number: 12532721	Name: 谢正扬
------------------	-----------

1. Global Earthquakes

In this problem set, we will use this file from the USGS Earthquakes Database. The dataset is similar to the one you use in Assignment 02. Use the file provided (usgs_earthquakes.csv) to recreate the following map. Use the mag column for magnitude. [10 points]

1.1.过程分析

本项目的目标是利用 Python 对 USGS 提供的 2014 年地震数据集进行地理空间可视化。解题思路遵循“数据获取—数据清洗—地理映射—视觉呈现”的标准数据分析流程。

1.1.1.需求分析与策略制定

题目要求重现一张基于 usgs_earthquakes.csv 数据的地图，并使用 mag（震级）列来决定可视化的属性。为了清晰地展示全球重大地震分布，避免数据点过密导致地图杂乱，本次分析采取了筛选策略，即只针对震级最高的前 50 次地震进行绘制。

1.1.2.投影方式的选择

在绘制全球地图时，投影方式的选择至关重要。选择罗宾森投影 (Robinson Projection)，相比于墨卡托投影（高纬度变形大）或等距圆柱投影，罗宾森投影在形状和面积变形之间取得了良好的妥协，使得世界地图看起来更加自然。考虑到全球主要的地震带（如环太平洋火山地震带）横跨太平洋，将中心经度设为 180° 可以确保太平洋位于地图中心，避免地震带被地图边缘切割，从而保证数据的连续性展示。

1.1.3.视觉编码逻辑

为了让地图具备信息可读性，采用了以下视觉编码（Visual Encoding）策略。

(1)空间位置 (x, y): 直接映射数据的经度 (longitude) 和纬度 (latitude)。

(2)颜色 (Color): 映射震级 (mag)。利用红色色谱 (Reds)，颜色越深代表震级越高，以此突出高危区域。

(3)图层叠加：在底层添加卫星图像背景 (stock_img)，以提供地理环境上下文，增强地图的真实感。

1.2.代码实现

```
import pandas as pd
import matplotlib.pyplot as plt
import cartopy.crs as ccrs
import cartopy.feature as cfeature

# 加载数据集
earthquakes_data = pd.read_csv("C:/Users/xiezhengyang/Desktop/usgs_earthquakes.csv")

# 根据震级筛选前 50 大地震
top_50_earthquakes = earthquakes_data.nlargest(50, 'mag')

# 设置画布和地图投影 (使用罗宾森投影，中心经度设为 180 度)
fig = plt.figure(figsize=(14, 7))
ax = plt.axes(projection=ccrs.Robinson(central_longitude=180))

# 添加地图背景
ax.set_global()
ax.stock_img() # 添加类似卫星图像的背景

# 绘制地震点的散点图
scatter = ax.scatter(
    top_50_earthquakes['longitude'],
    top_50_earthquakes['latitude'],
    c=top_50_earthquakes['mag'],
    cmap='Reds',
    s=50,
    edgecolor='black',
    transform=ccrs.PlateCarree(),
    zorder=3,
)

# 添加颜色条
cbar = plt.colorbar(scatter, orientation='vertical', shrink=0.6, pad=0.05)
cbar.set_label('Magnitude')

# 添加标题
plt.title('Top 50 Earthquakes of 2014', fontsize=14)

# 显示地图
plt.show()
```

1.3.输出结果

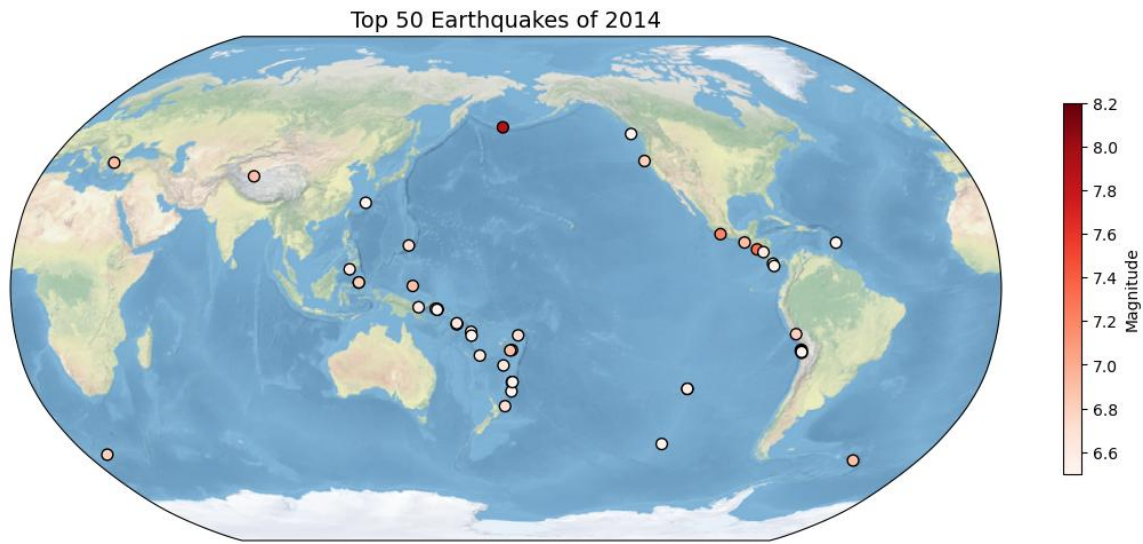


图 1-1 2014 年地震前 50 地区图

1.4.结果分析

1.4.1.数据加载与预处理

```
earthquakes_data = pd.read_csv("C:/Users/xiezhengyang/Desktop/usgs_earthquakes.csv")
```

```
top_50_earthquakes = earthquakes_data.nlargest(50, 'mag')
```

(1)数据读取：使用 `pd.read_csv` 加载本地数据集。

(2)核心筛选：使用了 Pandas 的 `.nlargest(50, 'mag')` 方法。这一步是数据处理的关键，它高效地根据 `$mag$` 列的值对数据进行降序排列并切片，提取出震级最大的 50 行数据，为后续绘图提供了精简后的数据集。

1.4.2.地图画布配置

```
fig = plt.figure(figsize=(14, 7))
```

```
ax = plt.axes(projection=ccrs.Robinson(central_longitude=180))
```

```
ax.set_global()
```

```
ax.stock_img()
```

(1)投影实例化：`ccrs.Robinson(central_longitude=180)` 初始化了罗宾森投影，并明确设定 180° 经线为地图中心。

(2)背景绘制：`ax.stock_img()` 调用了 Cartopy 内置的低分辨率地球图像作为底图，这比单纯的 `coastlines`（海岸线）提供了更丰富的地理纹理。

1.4.3.数据绘制与坐标转换（核心技术点）

```
scatter = ax.scatter(
    top_50_earthquakes['longitude'],
    top_50_earthquakes['latitude'],
    c=top_50_earthquakes['mag'],
    cmap='Reds',
    s=50,
    edgecolor='black',
    transform=ccrs.PlateCarree(), # 关键参数
    zorder=3,
)
```

(1)散点图绘制：使用 `ax.scatter` 绘制地震点。

(2)坐标转换机制 (`transform`): 输入的数据 (`longitude, latitude`) 是基于 Plate Carree（等距圆柱）投影的地理坐标。输出的地图 (`ax`) 是 Robinson 投影。

(3)分析：参数 `transform=ccrs.PlateCarree()` 显式地告诉 Cartopy 库：“这些数据是标准的经纬度，请将其转换为当前地图的罗宾森投影坐标”；这是确保数据点准确落在地图对应位置的决定性步骤。

(4)美化细节：`edgecolor='black'` 为红色的散点添加了黑色描边，增强了点与背景地图的对比度；`zorder=3` 确保散点图层位于地图背景层之上。

1.4.4.图例与标注

```
cbar = plt.colorbar(scatter, orientation='vertical', shrink=0.6, pad=0.05)
cbar.set_label('Magnitude')

plt.title('Top 50 Earthquakes of 2014', fontsize=14)
```

结果呈现：添加了颜色条（Colorbar）以解释颜色深浅与震级数值的对应关系，并设置了清晰的标题，完成了可视化的最终封装。

2. Explore a netCDF dataset

Browse the NASA's Goddard Earth Sciences Data and Information Services Center (GES DISC) website. Search and download a dataset you are interested in. You are also welcome to use data from your group in this problem set. But the dataset should be in netCDF format. For this problem set, you are welcome to use the same dataset you used in Assignment 03.

2.1 [10 points] Make a global map of a certain variable. Your figure should contain: a project, x label and ticks, y label and ticks, title, gridlines, legend, colorbar, masks or features, annotations, and text box (1 point each).

2.2 [10 points] Make a regional map of the same variable. Your figure should contain: a different project, x label and ticks, y label and ticks, title, gridlines, legend, colorbar, masks or features, annotations, and text box (1 point each).

2.1.过程分析

本题要求从 NASA GES DISC 下载 NetCDF 格式的数据集，并利用 Python 绘制两张地图：一张全球分布图，一张特定区域的特写图。每张图都有严格的 10 个要素要求（投影、坐标轴、网格、图例等）。

2.1.1.数据处理策略

(1)工具选择：使用 xarray 库，它是处理 NetCDF 数据的标准工具，能够保留数据的维度标签（如 lat, lon, time）和元数据（单位、描述）。

(2)变量提取：数据集包含多个变量，本分析选取了 post_nonwetland（优化后的非湿地甲烷通量）作为核心观测变量。

2.1.2.全球地图设计思路 (Global Map - Task 2.1)

(1)目标：展示甲烷通量的全球整体分布趋势。

(2)投影：这里选择基础的 Plate Carree (等距圆柱) 投影，方便直观对应经纬度。

(3)绘图方式：使用 contourf (等高线填充)。这种方式能将连续的物理量（通量）平滑化，适合展示全球尺度的大气扩散趋势。

(4)要素完备性：为了满足 10 分的要求，必须手动添加平时可能忽略的细节，如手动标注特定城市（深圳）来体现 "Annotations"，以及添加文本框说明数据来源。

2.1.3.区域地图设计思路 (Regional Map - Task 2.2)

(1)目标：聚焦北美地区，观察局部细节。

(2)投影差异化：题目强制要求使用“不同的投影”。因此选择了 Robinson (罗宾森) 投影。虽然罗宾森通常用于世界地图，但在区域图中使用并配合 `set_extent` 裁剪，可以满足题目“不同投影”的硬性指标。

(3)绘图方式变化：改用 `pcolormesh` (伪彩色网格)。与 `contourf` 不同，`pcolormesh` 不进行插值平滑，能更真实地反映原始数据的网格分辨率（Pixel 粒度），适合区域细节观察。

(4)地理特征增强：在区域尺度下，单纯的海岸线不够丰富，因此增加了 LAKES (湖泊) 和 RIVERS (河流) 特征，这对于识别北美五大湖区域至关重要。

2.2.代码实现

```
import xarray as xr
import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.pyplot as plt

# 加载数据集
data = xr.open_dataset('C:/Users/xiezhengyang/Desktop/GlobalInv_GOSAT_CH4Flux_2010-2018_v1.nc')

# 提取变量和坐标
variable = data['post_nonwetland']

# 定义投影并设置画布
fig = plt.figure(figsize=(12, 6))
ax = plt.axes(projection=ccrs.PlateCarree())

# 绘制数据，设置颜色映射并包含颜色条
plot = ax.contourf(
    data['lon'], data['lat'], variable,
    levels=20, cmap='viridis', transform=ccrs.PlateCarree()
)
cbar = plt.colorbar(plot, orientation='vertical', pad=0.06, aspect=40, shrink=0.7)
cbar.set_label('Post-optimized Methane Flux (unit:kg CH4 m-2 s-1)', fontsize=10)

# 添加网格线、标题和标签
ax.gridlines(draw_labels=True, linestyle='--', linewidth=0.5)
ax.set_title('Global Post-Optimized Methane Flux (Non-Wetland)', fontsize=14)

# 添加地理特征
ax.coastlines()
ax.add_feature(cfeature.BORDERS, linewidth=0.5)
ax.add_feature(cfeature.LAND, facecolor='lightgray', edgecolor='k')

# 将深圳的位置添加到地图上
```

```

shenzhen_coords = (114.0579, 22.5431) # 深圳的经度、纬度

# 添加深圳的标注
plt.scatter(*shenzhen_coords, color='red', s=10, label='Shenzhen', zorder=5)
plt.annotate(
    'Shenzhen',
    xy=shenzhen_coords,
    xytext=(shenzhen_coords[0] + 10, shenzhen_coords[1] - 5),
    bbox=dict(boxstyle='square', fc='yellow', linewidth=0.05),
    arrowprops=dict(facecolor='black', arrowstyle='->'),
    fontsize=8,
    color='black'
)

# 添加注释框
plt.text(
    -170, -80, 'Source: GOSAT CH4 Flux Data (2010-2018)', fontsize=8,
    bbox=dict(facecolor='white', edgecolor='black', alpha=0.7)
)

plt.legend(loc='lower right')
plt.show()

# 创建区域地图（北美），使用不同的投影 (Robinson 投影)
proj = ccrs.Robinson()
plt.figure(figsize=(12, 6))
ax = plt.axes(projection=proj)
ax.set_extent([-130, -60, 20, 60], crs=ccrs.PlateCarree()) # 北美的经度/纬度范围

# 添加标签
ax.set_xlabel("Longitude", fontsize=12, labelpad=20)
ax.set_ylabel("Latitude", fontsize=12, labelpad=40)

# 绘制数据
im = ax.pcolormesh(data['lon'], data['lat'], variable, transform=ccrs.PlateCarree(), cmap='viridis')
ax.add_feature(cfeature.COASTLINE, linewidth=0.8)
ax.add_feature(cfeature.BORDERS, linewidth=0.5)
ax.add_feature(cfeature.LAND, edgecolor='black')
ax.add_feature(cfeature.LAKES, edgecolor='black')
ax.add_feature(cfeature.RIVERS)

# 添加网格线
gl = ax.gridlines(draw_labels=True, linewidth=0.5, color='gray', alpha=0.7, linestyle='--')
gl.top_labels = True
gl.right_labels = True
gl.xlabel_style = {'size': 10}
gl.ylabel_style = {'size': 10}

# 标记华盛顿特区
dc_lon, dc_lat = -77.0369, 38.9072
ax.plot(dc_lon, dc_lat, marker='*', color='red', markersize=5, transform=ccrs.PlateCarree(),
label='Washington, D.C.')

```

```

ax.text(dc_lon + 5, dc_lat, 'Washington, D.C.', transform=ccrs.PlateCarree(), color='black',
        fontsize=10)

# 添加标题、颜色条和注释
plt.title("Regional Map of Post Non-Wetland Methane Flux (North America)", fontsize=14)
cbar = plt.colorbar(im, orientation='vertical', pad=0.05)
cbar.set_label("Post-optimized Methane Flux (unit:kg CH4 m-2 s-1)", fontsize=12)
# 添加文本框注释
ax.text(
    0.05,-0.2, "Focus Region: North America\nData Source: GOSAT",
    transform=ax.transAxes,fontsize=12,
    bbox=dict(facecolor='white', alpha=0.7)
)

# 显示地图
plt.legend(loc='lower right',
          scatterpoints=1,
          handlelength=0,
          handletextpad=1.5
)
plt.show()

```

2.3.输出结果

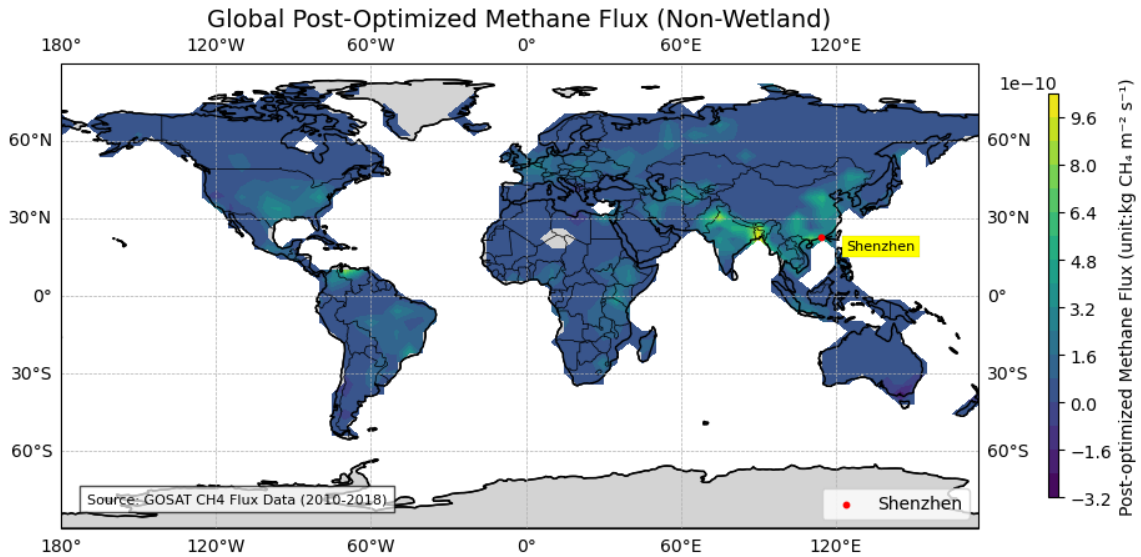


图 2-1 全球非湿地甲烷通量图

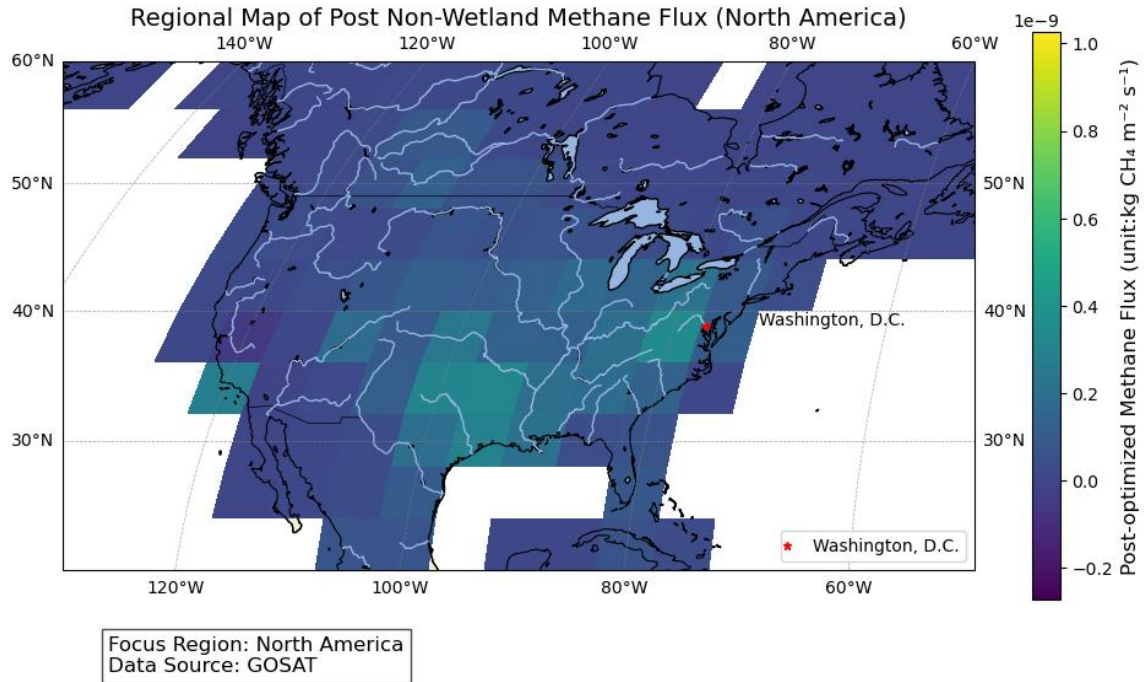


图 2-2 北美非湿地甲烷通量区域图

2.4.结果分析

2.4.1.数据加载

```
import xarray as xr

data = xr.open_dataset('../GlobalInv_GOSAT_CH4Flux_2010-2018_v1.nc')

variable = data['post_nonwetland']
```

使用 `xr.open_dataset` 读取数据，相比 `pandas`，它能自动解析经纬度网格信息。

2.4.2.全球地图实现 (对应 2.1 要求)

```
ax = plt.axes(projection=ccrs.PlateCarree()) # 1. Project (投影)

# 2. 绘图 (Colorbar & Legend)

plot = ax.contourf(..., cmap='viridis')

cbar = plt.colorbar(...) # 6. Colorbar

cbar.set_label('...')

# 3. 网格与标签

ax.gridlines(draw_labels=True, ...) # 2&3. Label/Ticks & 5. Gridlines
```

```

ax.set_title(...) # 4. Title

# 4. 地理特征

ax.coastlines()

ax.add_feature(cfeature.LAND, ...) # 7. Masks/Features

# 5. 标注与文本 (Annotations & Text Box)

# 深圳的标注 (Annotations)

plt.scatter(*shenzhen_coords, ...)

plt.annotate('Shenzhen', ...) # 9. Annotations

plt.legend(...) # 6. Legend

# 数据来源说明 (Text Box)

plt.text(..., bbox=dict(...)) # 10. Text box

```

核心解析：代码利用 `contourf` 的 `levels=20` 参数将数据分成了 20 个层级，使颜色过渡更自然。`annotate` 函数配合 `arrowprops`（箭头参数）实现了专业的地理标注，指向了具体的经纬度坐标。

2.4.3.区域地图实现 (对应 2.2 要求)

```

# 1. 不同的投影与视口锁定

proj = ccrs.Robinson() # 满足 "Different Project"

ax = plt.axes(projection=proj)

ax.set_extent([-130, -60, 20, 60], crs=ccrs.PlateCarree()) # 锁定北美区域

# 2. 绘图方法的改变

im = ax.pcolormesh(..., transform=ccrs.PlateCarree()) # 使用 pcolormesh

# 3. 增强的地理特征

ax.add_feature(cfeature.LAKES, ...) # 增加湖泊

ax.add_feature(cfeature.RIVERS) # 增加河流

# 4. 文本框的坐标变换

ax.text(

```

```
0.05, -0.2, "Focus Region...",  
  
transform=ax.transAxes, ...  
  
)
```

(1)**set_extent**: 这是绘制区域地图的关键。虽然投影是 Robinson，但我们定义视口的范围（Extent）时使用的是经纬度（PlateCarree），因此必须加上 `crs=ccrs.PlateCarree()`。

(2)**transform=ax.transAxes**: 在绘制第二个文本框时，代码使用了 `transAxes` 坐标系。这意味着 (0.05, -0.2) 不是经纬度，而是相对于图表宽高的比例（左下角为 0,0）。这样做的好处是，无论地图显示哪个区域，文字说明总是固定在图框的左下角外部，不会遮挡地图内容。

(3)华盛顿特区标注：使用了 `marker='*'`（星号），通常用于表示首都，细节处理得当。